

Programación II

Programación Orientada a Objetos

Abstracción · Encapsulamiento · Constructores y destructores ·

Actividad: el Incrementador

Hacer un programa que permita representar un Incrementador. Un incrementador es un número entero que comienza con un valor inicial y puede incrementarse a partir de un valor de incremento. También deberá llevar la cuenta de la cantidad de veces que incrementó.

Si arranca en 0 con incremento 50: **0** → **50** → **100** → **150** → ...

Restricciones

- El valor del incrementador no puede nunca disminuir.
- La cantidad de veces que se incrementó siempre debe aumentar de uno en uno. Tampoco puede disminuir.
- Siempre que el incrementador aumente debe hacerlo por la misma cantidad definida en el valor de incremento.

Primera solución: funciones y referencias

El estado vive en main y viaja por parámetro. Lo que se modifica va con &.

```
void incrementar(int &valor, int incremento,
               int &cantidad) {
    valor = valor + incremento;
    cantidad = cantidad + 1;
}

int main() {
    int valor, incremento, cantidad;
    inicializar(valor, incremento, cantidad, 0, 50);

    incrementar(valor, incremento, cantidad);
    cout << valor; // 50
}
```

Funciona, pero...

- valor y cantidad van por referencia: la función modifica las variables de main.
- El estado son tres variables sueltas que hay que arrastrar juntas en cada llamada.
- Cumplir las restricciones depende de la disciplina de quien programa.

El estado queda desprotegido

El estado es visible para todo el programa: cualquier línea puede romper las reglas.

```
int main() {  
    int valor = 0, incremento = 50, cantidad = 0;  
  
    incrementar(valor, incremento, cantidad);  
  
    incremento = 7; // ya no incrementa de a 50  
    valor = 0;      // el valor disminuyó  
    cantidad--;    // la cuenta miente  
}
```

Ninguna de esas tres líneas da error. El compilador no conoce las restricciones del enunciado.

Qué falta

- Nada protege al estado: es visible y modificable desde cualquier parte.
- Las reglas viven en la cabeza del programador, no en el código.
- Si el programa crece, el error aparece lejos de donde se definió el incrementador.

Programación Orientada a Objetos

Modelar el problema con objetos: datos y operaciones en una misma unidad, con reglas que el propio código hace cumplir.

Abstracción

Quedarse con lo esencial del problema y descartar todo lo demás.

La idea

- Un objeto se describe por lo que sabe (atributos) y por lo que hace (métodos).
- No modelamos todo el mundo real: sólo lo que el problema necesita.
- Del enunciado surgen, casi textualmente, los atributos y las operaciones.

Incrementador
valor : int incremento : int cantidadIncrementos : int
incrementar() incrementar(veces : int) getValor() : int getCantidadIncrementos() : int mostrarEstado()

qué sabe · qué hace

Encapsulamiento

Ocultar el estado y exponer sólo las operaciones que lo mantienen válido.

La idea

- **private**: nadie fuera de la clase puede leer ni escribir esos atributos.
- **public**: la interfaz que la clase ofrece al resto del programa.
- Las restricciones del enunciado pasan a ser responsabilidad de la clase.

public

incrementar() · getValor()
getCantidadIncrementos() · mostrarEstado()

private

valor
incremento
cantidadIncrementos

Al estado sólo se llega atravesando los métodos públicos.

La clase Incrementador

```
class Incrementador {  
private:  
    int valor;  
    int incremento;  
    int cantidadIncrementos;  
  
public:  
    void incrementar() {  
        valor = valor + incremento;  
        cantidadIncrementos++;  
    }  
  
    int getValor() const {  
        return valor;  
    }  
};
```

Qué cambió

- Desaparecen los &: los métodos ya no reciben el estado, lo tienen adentro.
- No hay setValor() ni setCantidadIncrementos(): si existieran, el valor podría disminuir.
- incrementar() es la única puerta de entrada, y siempre suma lo mismo y cuenta de a uno.

Encapsulamiento en acción

El mismo error, antes y después de encapsular.

Con funciones

```
valor = 0;  
incremento = 7;  
cantidad--;
```

Compila sin una sola advertencia.

Con la clase

```
inc.valor = 0;  
inc.incremento = 7;  
inc.cantidadIncrementos--;
```

error: 'valor' is a private member

Las restricciones dejaron de ser una promesa y pasaron a ser una propiedad del programa: el error se detecta al compilar, no cuando el usuario ve un número raro en pantalla.

Constructores

Método que se ejecuta al crear el objeto y lo deja en un estado válido.

Características

- Se llama igual que la clase y no devuelve ningún tipo, ni siquiera void.
- Se ejecuta automáticamente: no se invoca a mano.
- Puede haber varios (sobrecarga): con parámetros y por defecto.
- Es el lugar natural para validar los datos iniciales.

```
Incrementador(int inicial, int incr) {  
    valor = inicial;  
    if (incr <= 0) {  
        incr = 1; // el valor nunca baja  
    }  
    incremento = incr;  
    cantidadIncrementos = 0;  
}  
  
Incrementador() {  
    valor = 0;  
    incremento = 1;  
    cantidadIncrementos = 0;  
}
```

Destructores

Método que se ejecuta cuando el objeto deja de existir.

Características

- Se escribe con ~ delante del nombre de la clase.
- No recibe parámetros, no devuelve nada y no se sobrecarga: hay uno solo.
- Se dispara solo, al salir del ámbito o al hacer delete del objeto.
- Su trabajo típico es liberar lo que el objeto pidió prestado: memoria dinámica, archivos, conexiones.

```
class Historial {  
private:  
    int *registros;  
public:  
    Historial(int n) {  
        registros = new int[n]; // pide  
    }  
    ~Historial() {  
        delete [] registros; // devuelve  
    }  
};
```

El objeto se hace cargo de su propia memoria: quien lo usa no se olvida del delete.

De funciones a objetos

	Con funciones	Con objetos
Estado	Variables sueltas en main	Atributos privados del objeto
Acceso	Cualquier línea puede modificarlo	Sólo los métodos públicos
Reglas	Las recuerda quien programa	Las garantiza la clase
Inicio y fin	Se invoca inicializar() a mano	Constructor y destructor automáticos